

```
language=Python, basicstyle=, breaklines=true, keywordstyle=blue,  
commentstyle=gray, stringstyle=red, showstringspaces=false, frame=single,  
rulecolor=black!30
```

ECG Simulator

Comprehensive Technical Documentation

Phase 1/2 (Conduction Graph) and Phase 3 (FHN Cable)

Enrico Caruso

Engineering & Simulation

June 30, 2026

Contents

List of Figures

List of Tables

About This Documentation

This document provides exhaustive technical documentation of the ECG Simulator project, designed for four distinct audiences simultaneously:

- **Physicians and medical students:** ECG theory and clinical interpretation with minimal mathematical background required.
- **Bioscientists and physiologists:** Cardiac electrophysiology without requiring programming expertise.
- **Mathematicians and physicists:** Full mathematical rigor and physical modeling details.
- **Software engineers:** Complete implementation details, code structure, and dependencies.

All concepts are explained from first principles. Where appropriate, concepts are stratified so that readers can engage at their level of interest.

Chapter 1

Project Overview

1.1 Purpose and Scope

The ECG Simulator is a Python-based biomedical simulation framework that models the electrical activity of the human heart and generates realistic 12-lead electrocardiograms (ECGs) under normal and pathological conditions. The simulator serves as both a:

1. **Educational tool:** Teaching ECG interpretation and understanding cardiac electrophysiology
2. **Research platform:** Studying the relationship between conduction abnormalities and ECG morphology
3. **Clinical decision support:** Generating reference ECGs for various pathological states

1.2 Key Capabilities

The simulator can model the following clinical scenarios:

- Normal sinus rhythm (NSR) at various heart rates
- Atrial pathologies: atrial fibrillation, atrial flutter, atrial premature contractions
- Atrioventricular (AV) conduction abnormalities: first-, second-, and third-degree AV blocks
- Bundle branch blocks: right bundle branch block (RBBB), left bundle branch block (LBBB)
- Ventricular arrhythmias: premature ventricular contractions (PVCs), ventricular tachycardia

- Accessory pathway conduction: Wolff-Parkinson-White (WPW) syndrome
- Sinus node dysfunction: sinus bradycardia, sinus tachycardia

1.3 Development Phases

The project is organized into three phases:

1.3.1 Phase 1/2: Conduction Graph Engine

The Phase 1/2 implementation uses a **discrete conduction graph** model where:

- The heart is represented as a directed graph of **nodes** (anatomical regions: SA node, RA, LA, AV node, His bundle, ventricular segments)
- **Activation propagates** as breadth-first search (BFS) with physiological conduction velocities
- Each node has fixed **activation delay** from the previous node
- Refractory periods are **hardcoded** for each node
- **ECG dipole contributions** from each node are modeled as Gaussian functions in time

Advantages:

- Extremely fast (microseconds per step)
- All pathologies implemented as parameter overrides
- Clinically calibrated timing and morphology

Limitations:

- Refractory period is an input, not emergent from dynamics
- No continuous state evolution or tissue coupling

1.3.2 Phase 3: FitzHugh-Nagumo Cable Engine

The Phase 3 implementation uses a **1-D monodomain FitzHugh-Nagumo cable** model where:

- The conduction system is represented as a **linear chain of cells** (18 total)

- Each cell solves the **FitzHugh-Nagumo ordinary differential equations** (ODEs)
- Action potential (AP) propagates via **diffusion coupling** between adjacent cells
- Refractory period **emerges** naturally from the slow recovery variable dynamics
- **Multi-beat rhythm** requires no explicit pacemaker programming
- ECG dipoles updated *continuously* as the AP wavefront propagates

Advantages:

- Emergent refractory period (no hardcoding)
- Continuous state evolution (more physiologically faithful)
- Can model effects of rate-dependent APD changes

Limitations:

- Slower than Phase 1/2 ($2.2\times$ real-time on modern hardware)
- Pathologies less developed (manual parameter override)
- Timing requires careful calibration

1.4 User-Selectable Engine

The user can switch between engines via a toolbar combo box. The switch:

- Is seamless (pause current engine, swap, resume)
- Reinitializes the new engine with current parameter values
- Does not modify parameter settings

Chapter 2

Theoretical Foundations: Mathematical and Physical Models

2.1 Cardiac Electrophysiology Primer

2.1.1 The Action Potential

Every cardiac myocyte is electrically excitable. When stimulated, it generates an **action potential** (AP)—a stereotypical time-evolution of transmembrane voltage V that drives heart contraction.

Phases of the Cardiac Action Potential

The AP has five distinct phases (Figure 2.1):

[Diagram - text description in document]

Figure 2.1: Five phases of the cardiac action potential. Phase 0: rapid depolarization (inward Na^+ current). Phase 1: early repolarization (outward K^+ current). Phase 2: plateau (inward Ca^{2+} current balances outward K^+). Phase 3: rapid repolarization. Phase 4: resting state.

Phase 0 (Depolarization): The membrane potential V rapidly depolarizes from resting $V_{\text{rest}} \approx -90$ mV to peak $\approx +30$ mV due to rapid inward Na^+ current. This phase lasts ≈ 1 ms.

Phase 1 (Early Repolarization): Brief outward K^+ current causes V to decline slightly.

Phase 2 (Plateau): Inward Ca^{2+} current balances outward K^+ current, maintaining $V \approx 0$ mV. This phase lasts 100–300 ms depending on cell type and conditions.

Phase 3 (Late Repolarization): Dominant outward K^+ current repolarizes the cell back toward V_{rest} .

Phase 4 (Resting/Diastole): The cell remains at rest potential, ready for the next stimulus.

Refractoriness

During and immediately after an AP, the cell is **refractory**—it cannot be re-excited by an incoming stimulus:

- **Absolute refractory period:** During phases 0–2 and early phase 3, Na^+ channels are inactivated; no re-stimulation possible.
- **Relative refractory period:** Late phase 3, when V is still below threshold. Re-stimulation is possible only with a **supernormal stimulus** (larger than normal).

The duration of refractoriness is roughly proportional to the AP duration (**APD**). Longer APD $\Rightarrow$ longer refractoriness.

2.1.2 Conduction Velocity

Electrical activation spreads from cell to cell via gap junctions (protein channels connecting adjacent myocytes). The speed of this propagation is the **conduction velocity** (CV):

$$\text{CV} \approx 0.5 \sqrt{\frac{D}{\epsilon}} \cdot \frac{\text{dx}}{\text{cell time constant}}$$

where:

- D is the diffusion coefficient (cytoplasmic resistivity)
- ϵ is the excitability (inversely related to refractory period)
- dx is the cell size

Typical values in the heart:

- Atrial myocardium: 0.5–1.0 m/s
- His-Purkinje system: 2–4 m/s (high CV)
- Ventricular myocardium: 0.3–0.5 m/s
- AV node: 0.02–0.05 m/s (slowest, with programmed delay)

2.2 ECG Generation: The Dipole Model

2.2.1 Cardiac Electrical Dipole

At any instant, the heart has a net electrical dipole moment $\mathbf{p}(t)$ pointing from negative to positive charge:

$$\mathbf{p}(t) = \sum_{\text{all myocytes}} q_i(t) \mathbf{r}_i$$

where $q_i(t)$ is the charge state of myocyte i at position $\mathbf{r}_i$.

During the AP:

- **Depolarization phase 0:** Wavefront of depolarization creates a dipole pointing from rest (negative) to depolarized (positive)
- **Plateau phase 2:** Little dipole moment (most of heart is depolarized)
- **Repolarization phase 3:** Wavefront of repolarization creates a dipole pointing from depolarized (negative) to rest (positive)

2.2.2 12-Lead ECG and Lead Vectors

Each of the 12 ECG leads measures the projection of the cardiac dipole onto a fixed spatial direction called the **lead vector**.

The 12 leads comprise:

- **Limb leads:** I, II, III (measure frontal plane: left-right and inferior-superior)
- **Augmented leads:** aVR, aVL, aVF (also frontal plane, normalized)
- **Precordial leads:** V1–V6 (measure transverse plane: anterior-posterior and left-right)

Standard lead vectors (in mV per mV of dipole component) are fixed by convention and embedded in the simulator.

2.2.3 Gaussian Dipole Model

In this simulator, each anatomical node (e.g., RV, LV lateral) contributes to the ECG as a **time-localized dipole**:

$$\mathbf{p}_{\text{node}}(t) = \mathbf{d}_{\text{node}} [g_{\text{depol}}(t - t_{\text{act}}) + A_{\text{repol}} g_{\text{repol}}(t - t_{\text{act}})] \quad (2.1)$$

where:

- $\mathbf{d}_{\text{node}}$ is the **dipole vector** (anatomy-specific, units: mV-m)
- t_{act} is the **activation time** (when the node first crossed threshold)
- $g_{\text{depol}}(t)$ is the **depolarization Gaussian**: $g_d(t) = \exp\left(-\frac{t^2}{2\sigma_d^2}\right) \Theta(t)$ (active for $t > 0$)
- $\sigma_d \approx 12.7$ ms (controls QRS width; Phase 1/2 value)
- $g_{\text{repol}}(t)$ is the **repolarization Gaussian**: $g_r(t) = \exp\left(-\frac{(t-\tau_r)^2}{2\sigma_r^2}\right)$ (peaks at repolarization delay $\tau_r \approx 280$ ms)
- $A_{\text{repol}} \approx -0.3$ (T-wave is opposite polarity to QRS)

The total voltage in lead k is the sum over all nodes:

$$\text{ECG}_k(t) = \sum_{\text{nodes}} \mathbf{L}_k \cdot \mathbf{p}_{\text{node}}(t) \quad (2.2)$$

where $\mathbf{L}_k$ is the **lead vector** for lead k (dimensionless, normalized).

2.3 Model (Phase 3)

2.3.1 Biological Motivation

The model is a two-variable simplified model of the Hodgkin-Huxley equations that captures the essential fast-slow dynamics of cardiac action potentials without explicitly modeling individual ion channels.

2.3.2 The FitzHugh-Nagumo Equations

In normalized units, the 1-D cable equations are:

$$\frac{\partial v}{\partial t} = \frac{D}{\tau} \frac{\partial^2 v}{\partial x^2} + \frac{1}{\tau} \left(v - \frac{v^3}{3} - w + I_{\text{ext}} \right) \quad (2.3)$$

$$\frac{\partial w}{\partial t} = \frac{\varepsilon}{\tau} (v + a - b \cdot w) \quad (2.4)$$

where:

- $v(x, t)$ is the **transmembrane voltage** (normalized: $-1.2 \leq v \leq 2$; unphysiological units for mathematical simplicity)
- $w(x, t)$ is the **slow recovery variable** (models inactivation of inward current and activation of outward current)

- x is the position along the cable
- D is the **diffusion coefficient** (cell-to-cell coupling)
- τ is the **physical time scale** (all times scale inversely with τ)
- ε is the **time scale separation factor** (small $\varepsilon \Rightarrow$ slow w recovery)
- a, b are constants ($a \approx 0.7, b \approx 0.8$)
- I_{ext} is external stimulus current (applied at specific locations during appropriate times)

Fast-Slow Dynamics

The key insight is that v evolves on the **fast timescale** $\sim \tau$, while w evolves on the **slow timescale** $\sim \tau/\varepsilon$:

- **Depolarization (Phase 0)**: v rapidly increases as the depolarizing current dominates
- **Plateau (Phase 2)**: v is high, w slowly increases, gradually opposing the depolarizing current
- **Repolarization (Phase 3)**: Once w is large enough, it repolarizes v back toward rest
- **Refractory period**: w remains large, suppressing v from re-depolarizing. As w slowly decays, v can eventually respond again

Action Potential Duration

The AP duration (APD) is determined by the recovery timescale:

$$\text{APD} \sim \frac{\tau}{\varepsilon \cdot b}$$

For with $\tau = 15$ ms, $\varepsilon = 0.08$, $b = 0.8$:

$$\tau_w = \frac{0.015}{0.08 \times 0.8} = 0.234 \text{ s} = 234 \text{ ms}$$

This recovery time constant drives the refractoriness: ventricular cells take ≈ 234 ms to fully recover. At 70 bpm (cycle length = 857 ms), a cell fires, then 857 ms later is stimulated again when it has recovered $857/234 \approx 91\%$ of the way. Multi-beat ECGs therefore emerge naturally.

2.3.3 Discretization: Monodomain Cable

The cable is discretized into N_{cells} compartments. The Laplacian $\frac{\partial^2 v}{\partial x^2}$ is approximated by finite differences:

$$\left. \frac{\partial^2 v}{\partial x^2} \right|_i \approx \frac{v_{i-1} - 2v_i + v_{i+1}}{\Delta x^2}$$

with no-flux (Neumann) boundary conditions at the ends:

$$v'_0 = v_1 - v_0, \quad v'_{N-1} = v_{N-2} - v_{N-1}$$

2.3.4 Numerical Integration

The discretized system is integrated using explicit Euler with timestep $\Delta t = 0.1$ ms:

$$v_i^{n+1} = v_i^n + \Delta t \left[\frac{D}{\Delta x^2 \tau} (v_{i-1}^n - 2v_i^n + v_{i+1}^n) + \frac{1}{\tau} \left(v_i^n - \frac{(v_i^n)^3}{3} - w_i^n + I_i^n \right) \right] \quad (2.5)$$

$$w_i^{n+1} = w_i^n + \Delta t \frac{\varepsilon}{\tau} (v_i^n + a - b \cdot w_i^n) \quad (2.6)$$

The CFL (Courant-Friedrichs-Lewy) stability condition requires:

$$D \cdot \frac{\Delta t}{\tau \Delta x^2} \leq 0.5$$

For the current values ($D_{\text{max}} = 72$, $\Delta t = 0.1$ ms, $\tau = 15$ ms, $\Delta x = 1$):

$$\text{CFL} = 72 \times \frac{0.0001}{0.015 \times 1^2} = 0.48 < 0.5 \quad \checkmark$$

2.4 Conduction Graph Model (Phase 1/2)

2.4.1 Graph Structure

The heart is modeled as a directed acyclic graph (DAG) with fixed **nodes** (anatomical regions) and fixed **edges** (conduction pathways):

$$\text{Graph} = (V, E) \quad (2.7)$$

where:

- $V = \{\text{SA, RA, LA, AV node, His, LBB, RBB, SEPT, LV, RV, etc.}\}$ (13 nodes in total)
- $E = \{(u, v) : \text{activation can propagate from } u \text{ to } v\}$

2.4.2 Activation Propagation

At each heartbeat, the node fires, triggering a **breadth-first search (BFS)** wave propagation:

1. Queue $\leftarrow$ [SA node]
2. Activation times $t_{\text{act}} \leftarrow$
3. While Queue is not empty:
 - (a) $u \leftarrow$ Dequeue
 - (b) For each neighbor v of u in Graph:
 - i. If v not yet activated:
 - ii. Delay $\Delta t_{\text{edge}} \leftarrow$ conduction delay on edge (u, v)
 - iii. Refractory check: if $(t_{\text{current}} - t_{\text{act}}[v]) \leq$ refractory period of v , skip
 - iv. Otherwise: $t_{\text{act}}[v] \leftarrow t_{\text{act}}[u] + \Delta t_{\text{edge}}$
 - v. Enqueue v

Each node has a **refractory period** (hardcoded, not emergent):

- , atrial nodes: 300 ms (atrial refractoriness)
- node: 300 ms
- Ventricular nodes: 250 ms

The **conduction delay** on each edge is also fixed:

- SA to RA: 0 ms (instantaneous)
- RA to LA: 100 ms
- LA to AV node: 50 ms
- AV node programmed delay: 100 ms (controlled by parameter)
- AV to His: 0 ms
- His to ventricular nodes: node-dependent (0–50 ms)

2.4.3 Pathological Modifications

Pathologies are modeled by **dynamically modifying** the graph structure or conduction parameters:

- **AV Block III°**: Remove all edges from LA to AV node
- **RBBB**: Increase RBB conduction delay by 50 ms
- **Sinus bradycardia**: Increase SA cycle length
- **Atrial fibrillation**: Replace BFS with chaotic re-entrant activation

Chapter 3

Implementation in Python: Core Modules and Functions

This chapter describes the Python implementation of all models in the simulator.

3.1 Project Directory Structure

```
cardiac_sim/+-core/Corecomputationlayer|+-cell_models/Ionicmodels(FHN, Aliev-  
Panfilov, etc.)|+-conduction/Conductiongraph(Phase1/2)|+-tissue/Cablemodel(Phase3)|+  
--ecg/ECGgeneration(leadfields, dipoles)|+-interfaces.pyAbstractbaseclasses|+  
--parameter_model.pyParameterdefinitionsandstorage|--simulation_worker.pyWorkerthreadforsim  
--simulation/High-levelsimulationengines|+-engine.pyAbstractSimulationEngine|+  
--graph_engine.pyPhase1/2implementation|+-cable_engine.pyPhase3implementation|+  
--solver.pyNumericalsolvers(stub)+-gui/PyQt6GUIlayer|+-main_window.pyMainwindow, tool  
--ecg_display.pyECGwaveformplotter|+-parameter_panel.pyParameterstreewidthget|+  
--pathology_panel.pyPathologyselector+-plugins/Pathologyplug-insystem|+-  
av_blocks.pyAVblockpathologies|+-bundle_blocks.pyBundlebranchblocks|+-atrial.pyAtrialpatholo  
--ventricular.pyVentricularpathologies|--sinus_rhythms.pySAnodedisorders--main.pyApplicati
```

3.2 Core Interfaces and Abstract Classes

3.2.1 SimulationEngine Abstract Base Class

File: cardiac_sim/core/interfaces.py

```
class SimulationEngine(ABC): @abstractmethod def initialize(self, params: Simula-  
tionParameters) -> None: """Initialize engine with parameters.""" pass  
    @abstractmethod def step(self, dt_s : float) -> ECGSample : """Advancebydt_sseconds, returnECG  
    @abstractmethod def apply_pathology(self, plugin : AbstractPathologyPlugin) ->  
    None : """Applyapathologicalmodification.""" pass
```

```
@abstractmethod def get_state(self) -> SimulationState : """Get current simulation state(HR, time)
```

All engines (graph, cable) inherit from this abstract class and must implement all methods.

3.2.2 ECGSample and SimulationState Data Classes

```
@dataclass class ECGSample: timestamp: float Current time in seconds leads: np.ndarray
Shape (12,) with voltages in mV
```

```
@dataclass class SimulationState: time: float heart_rate : float BPM is_running : bool
```

3.3 Implementation: ConductionGraphEngine

File: cardiac_sim/simulation/graph_engine.py (400 lines)

3.3.1 Class Overview

```
class ConductionGraphEngine(SimulationEngine): def __init__(self): self.params: SimulationParameters = None self.nodes: dict[str, ECGNode] = {}
def initialize(self, params: SimulationParameters) -> None: self.params = params self.nodes = build_nodes() from conduction/graph.py self.graph = build_physiological_graph() self.reset()
def step(self, dt_s : float) -> ECGSample : self.time += dt_s
Check if SA should fire (based on HR) if self.time >= self.next_fire : self.fire_sa()
Compute ECG from active beat ecg = self.compute_ecg(self.time) return ECGSample(timestamp = self.time, leads = ecg)
def fire_sa(self) -> None : BFS propagation from SA node self.beat = self.run_bfs() self.next_fire = self.params.sa_node.cycle_length_ms / 1000.0
def run_bfs(self) -> BeatRecord : Breadth-first search to compute activation times Returns a BeatRecord
def compute_ecg(self, t_s : float) -> np.ndarray : Sum Gaussian dipoles from all active nodes...
```

3.3.2 Key Functions

_build_nodes and build_physiological_graph

File: cardiac_sim/core/conduction/graph.py

These functions construct the fixed conduction graph and node parameters:

```
def build_nodes() -> dict[str, ECGNode] : """Return dict of anatomical nodes with ECG dipole parameters
"SA_NODE" : ECGNode(dipole_vector = [-1.0, 0.5, 0.2], mV - m, normalized depolarization = 0.010, 1000)
def build_physiological_graph() -> dict[str, list[str]] : """Return adjacency list of conduction pathway
```

_compute_ecg

Sums Gaussian dipoles (Eq. 2.1) over all nodes:

```

def compute_ecg(self, t_s : float) -> np.ndarray : dipole_total = np.zeros(3) 3Ddipolevector
if self.beat_isnotNone : for node_name, t_act in self.beat.activation_times.items() : node =
self.nodes[node_name]
    Depolarization Gaussian g_depol = np.exp(-(t_s - t_act) ** 2 / (2 * node.depol_duration *
** 2)) if t_s < t_act : g_depol = 0
    Repolarization Gaussian (peaks at t_act + repol_delay) t_repol_peak = t_act + node.repol_delay g_repol =
np.exp(-(t_s - t_repol_peak) ** 2 / (2 * node.repol_duration ** 2))
    Total dipole contribution dipole = node.dipole_vector * (g_depol - node.repol_amplitude *
g_repol) dipole_total += dipole
    Project onto 12 lead vectors ecg_12lead = self.lead_field.project(dipole_total).shape(12,) return ecg_12lead

```

3.4 Implementation: ConductionCableEngine

File: cardiac_sim/simulation/cable_engine.py (450 lines)

3.4.1 Class Overview

```

class ConductionCableEngine(SimulationEngine):
    def __init__(self, self.cable: ConductionCable1D = None, DFHN_cable: self.nodes =
    def initialize(self, params: SimulationParameters) -> None:
        self.params = params
        self.cable = ConductionCable1D(params)
        self.nodes = build_cable_nodes()
        adjusted_gaussians = self.time = 0.0
    def step(self, dt_s : float) -> ECGSample :
        self.time += dt_s
        SA_pacemaker if self.time >= self.next_safire : self.fire_safire()
        Always advance cable (to keep internal clock synchronized)
        newly = self.cable.advance(dt_s)
        If beat in progress, collect nodes as they activate
        if self.beat_inprogress : for node, t_act in newly.items() :
            if node not in self.current_beat_nodes :
                self.current_beat_nodes[node] = t_act
            KEYFIX : update active beat time
            self.active_beats[-1].activation_times[node] = t_act
        Commit beat when done ... (beat completion logic)
        Compute ECG from all active beats
        ecg = self.compute_ecg(self.time)
        return ECGSample(timestamp=self.time, leads=ecg)

```

3.4.2 Cable Propagation: ConductionCable1D

File: cardiac_sim/core/tissue/cable_1d.py (500 lines)

Class Overview and Fixed Parameters

Fixed FHN model parameters (Phase 3-D calibrated)

$$a_A = 0.700 \text{ FHN parameter } a_B = 0.800 \text{ FHN parameter } b_{EPS} = 0.080 \text{ recovery rate } T_{AU} = 0.015 \text{ timescale } (15 \text{ ms for correct PR/QR stimulation})$$

$$D_{T_{INT}} = 0.0001 \text{ internal micro-step } (0.1 \text{ ms})$$

Cable structure ${}_NA = 8atrialcells(SA+7){}_NV = 10ventricularcells(His + LBB(3) + LV(6))$

Diffusion coefficients ${}_DATR = np.array([0.0, 2.88, 2.88, 2.88, 2.88, 2.88, 2.88, 2.88]){}_DVEN = np.array([0.0, 72.0, 72.0, 72.0, 3.0, 3.0, 3.0, 3.0, 3.0, 3.0])$

Stimulus ${}_STIM_AMP = 1.5suprathreshholdcurrent{}_STIM_DUR = 0.01010ms$

```
class ConductionCable1D: def init(self, params: SimulationParameters): self.s = np.ndarray(({}_NA, 2))(cells, [v, w]) self.sv = np.ndarray(({}_NA, 2))
def new_beat(self, sa_fire_time : float, params : SimulationParameters) -> None :
"""Reset for a new beat at sa_fire_time.""" self.t = sa_fire_time self.s_time_end_a = sa_fire_time + {}_STIM_DUR
self.sv_ending_a = math.inf Reset activation trackers self.ta_ct_a[:] = math.inf self.ta_ct_v[:] = math.inf self.ta_ct_a[0] = sa_fire_time
def advance(self, dt_s : float) -> dict[str, float] : """Integrate by dt_s seconds, return newly activated nodes
max(1, round(dt_s / {}_DT_INT)) dt_micro = dt_s / n_micro newly : dict[str, float] =
for i in range(n_micro) : self.t += dt_micro self.s_step_atrial(dt_micro) self.s_step_ventricular(dt_micro)
Check for threshold crossings for node, (i0, i1) in {}_ATTRS_EGS.items() : (threshold_detection_logic)...
for node, (i0, i1) in {}_VEN_EGS.items() : (threshold_detection_logic)...
return newly
def s_step_atrial(self, dt : float) -> None : """One FHN Euler step on atrial cable.""" V =
np.ascontiguousarray(self.s_a[:, 0]) W = np.ascontiguousarray(self.s_a[:, 1]) I = np.zeros({}_NA) if self.t < self.s_time_end_a :
I[0] = {}_STIM_AMP V_new, W_new = f_hn_cable_step(V, W, {}_DATR, I, dt, {}_TAU_A, {}_TAU_B, {}_EPS) self.s_a[:, 0] = V_new self.s_a[:, 1] = W_new
```

Numerical Integration: `_fhn_cable_step`

File: `cardiac_sim/core/tissue/cable_1d.py`

```
def f_hn_cable_step(V, W, D, I_ext, dt, tau_s, a, b, eps) : """Single explicit Euler step for FHN cable.
Implements equations (1) and (2) with finite-difference Laplacian. """ Compute Laplacian with no-flux BCs
lap = np.empty_like(V) lap[1 : -1] = V[1 : -2] - 2.0 * V[1 : -1] + V[2 : ] lap[0] = V[1] - V[0] lap[-1] = V[-2] - V[-1]
inv_tau = 1.0 / tau_s dv = (D * lap + V - V * 3 / 3.0 - W + I_ext) * inv_tau dw = eps * (V + a - b * W) * inv_tau
return V + dt * dv, W + dt * dw
```

This function is Numba-JIT compilable (if compatible NumPy version) for $10\text{-}20\times$ speedup.